

SQL Schema Generator User Guide

This custom ORM tool uses Java Annotations and Reflection to automatically generate PostgreSQL-compatible SQL CREATE TABLE statements from your Java Entity classes. This guide covers how to set up your entity classes, how the annotations work, and how to execute the generation process.

1. Core Annotations

These annotations mark your classes and fields to instruct the generator on how to build the database schema.

Class Level

- **@DbTable(name = "custom_name")**
 - **Usage:** Optional. Use this to override the default table name (which is the lowercase class name).
 - **Example:**
`@DbTable(name = "app_users")`
`public class User { ... }`

Field Level (Columns)

- **@DbId**
 - **Usage:** Marks the Primary Key.
 - **Logic:**
 - Long → BIGSERIAL PRIMARY KEY
 - Integer → SERIAL PRIMARY KEY
 - UUID → UUID PRIMARY KEY
 - **Note:** If a field is named id (case-insensitive), it is automatically treated as a Primary Key even without this annotation.
- **@DbColumn(name, nullable, unique)**
 - **Usage:** Customizes column properties.
 - **Attributes:**
 - name: Override column name (default is field name).
 - nullable: Set false for NOT NULL constraint (default true).
 - unique: Set true for UNIQUE constraint (default false).
 - **Example:**
`@DbColumn(name = "full_name", nullable = false)`
`private String name;`

Field Level (Relationships)

- **@DbOneToOne**
 - **Effect:** Creates a Foreign Key column (fieldName_id) with a UNIQUE constraint.
- **@DbManyToOne**
 - **Effect:** Creates a standard Foreign Key column (fieldName_id).
- **@DbOneToMany**
 - **Effect:** Ignored in the current table (the FK is on the child side), but adds a SQL comment for clarity.
- **@DbManyToMany**
 - **Effect:** Creates a separate Join Table (TableA_TableB) with two Foreign Keys.

2. How to Create an Entity

1. Extend a Base Class (Optional but recommended):

If you use Entity<ID>, the generator scans parent classes, so id fields in the parent are correctly included.

2. Add Fields & Annotations:

```
@DbTable(name = "employees")
public class Employee {

    @DbId
    private Long id; // -> BIGSERIAL PRIMARY KEY

    @DbColumn(nullable = false)
    private String name; // -> VARCHAR(255) NOT NULL

    @DbColumn(unique = true)
    private String email; // -> VARCHAR(255) UNIQUE

    @DbManyToOne
    private Department department;
    // -> department_id BIGINT REFERENCES department(id)
}
```

3. Supported Types:

- String → VARCHAR(255)
- int / Integer → INT
- long / Long → BIGINT

- double / Double → DOUBLE PRECISION
- boolean / Boolean → BOOLEAN
- LocalDate → DATE
- LocalDateTime → TIMESTAMP
- Enum → VARCHAR(50)

3. Running the Generator

The logic is contained in `SqlSchemaGenerator.java`.

1. **Configure Output Path:**

Ensure the path in `main()` points to your desired location (default is `src/main/resources/db/schema.sql`).

2. **Register Entities:**

Add your Entity classes to the entities list in `main()`. **Order matters** if you have relationships!

- Create Parent tables **before** Child tables (tables with Foreign Keys).

```
List<Class<?>> entities = List.of(
    Department.class, // Create this first (referenced by Employee)
    Employee.class   // Has FK to Department
);
```

3. **Run the main method:**

- It generates the SQL.
- It writes to `schema.sql`.
- It calls `DatabaseConnection.resetSchema()` to **wipe and recreate** your database immediately.

4. Important Notes

- **Inheritance Strategy:** The generator uses a "**Table** per Concrete **Class**" approach but includes fields from all parent classes (via recursion). It does *not* create separate tables for abstract parents unless you explicitly include them in the list.
- **Static Fields:** static fields are automatically ignored.
- **Heuristics:** If a field name contains "email", "isbn", "cnp", or is "username", it is automatically marked UNIQUE even without the annotation.
- **Database Reset:** The generator calls `DatabaseConnection.resetSchema()`, which executes `DROP SCHEMA public CASCADE`. ****Do not run this on a**